

such a mechanism in place, it is feasible to build mobile applications for store or middleware applications that act as a bridge between Spree and a warehouse or inventory system.

Great support!

Spree has a vibrant community backing the ecosystem. For organisations that need hand-holding with the entire set-up, cost-effective paid support plans are easily available.

The structure of gems

Spree itself is a collection of gems that provide all the functionality. In the Spree Git repository, the following gems are present:

GEM	Functionality
<i>spree_api</i>	Provides REST API access
<i>spree_auth</i>	Provides authentication and authorisation
<i>spree_cmd</i>	Utility for installing Spree and creating extensions
<i>spree_core</i>	Simple overview dashboard
<i>spree_dash</i>	Simple overview dashboard
<i>spree_promo</i>	Coupons and promotions
<i>spree_sample</i>	Sample data and images

Rails engines, since their introduction in Rails 3, let developers build mini applications that provide additional functionality to host applications. All the constituent gems are nothing but Rails 3 engines.

Trying out Spree

The Spree creators have a live demo running at <http://demo.spreecommerce.com/> to showcase a sample Spree store without admin access. If you want a personal sandbox with full access, you need to sign up at http://spreecommerce.com/one_click/signup, after which a personal store is created, and login credentials are mailed to you.

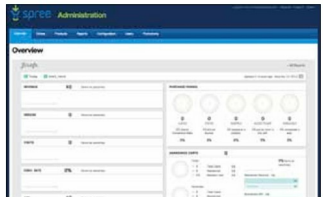
For folks who love to get their hands dirty, installing Spree locally on your machine requires very little effort, but some technical chops. Without further ado, let's get started with installing Spree. For the sake of brevity, assume a system running Ruby on RVM (Ruby Version Manager). First, install the latest Rails gem with `gem install rails -v 3.2.3`; next, install the Bundler gem with `gem install bundler`. A very popular image-manipulating library, ImageMagick, is used by Spree to manipulate images. If you want to add product images in your store, you need to install this suite. ImageMagick installation is OS-dependent and a quick Web search will point you to some excellent resources. Finally, install the Spree gem with `gem install spree` and you're done with the first part.

Now, you need to create a Rails application that makes use of Spree, with, for example, `rails new mythstore`; after that, change to the application directory (`cd mythstore`) and install Spree for this application (`spree install`). Spree will ask you some questions regarding the installation of sample data. Choose based on your requirements, or hit 'Enter' to go with the default options.

With Rails 3.1, an 'Asset Pipeline' was introduced to the



Demo



Demo1

Rails framework in order to concatenate and minimise client-side assets such as JavaScript and CSS files. It also allows developers to write assets in languages like Sass or CoffeeScript. This feature can cause some performance issues in development mode. To fix this, issue the following *precompile* task:

```
bundle exec rake assets:precompile:nondigest
```

You are now ready to take Spree for a spin! Start a Web server on your local machine by executing `rails s`, then fire up your browser and point it to `http://localhost:3000` to see the Web store.

Beyond the horizon

Spree comes bundled with a lot of documentation. To deploy a production Spree store instance on the Web, you can dig deeper and read the entire documentation at <http://guides.spreecommerce.com/>. To check out some real-world stores that are already running Spree, visit <http://spreecommerce.com/resources/showcase>. Seasoned Rails developers can read the contribution guide at http://guides.spreecommerce.com/contributing_to_spree.html to add their efforts to this amazing project. **END**

By: Nitish Upreti

The author is a Rubyist at heart. A technology start-up enthusiast and classic rock lover, he tweets as @nitish and blogs at <http://niti.sh>.